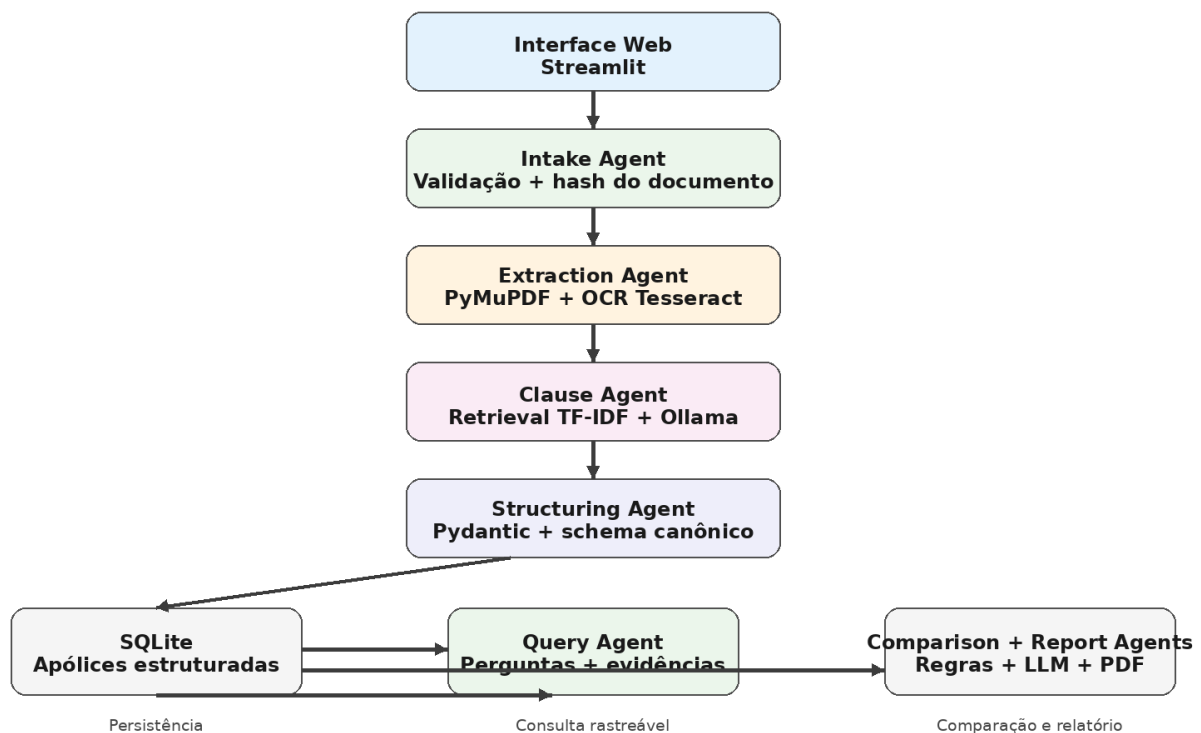


POLICYCOMPARE AI

Plataforma Inteligente para Análise e Comparação de Apólices

Relatório Técnico - MVP

PolicyCompare AI - Arquitetura do MVP



Versão de demonstração - Outubro de 2026

1. Resumo executivo

O PolicyCompare AI é um MVP local para automatizar etapas repetitivas da leitura e comparação de apólices de seguro. A solução recebe documentos em PDF ou imagem, extrai o conteúdo, identifica informações relevantes por meio de IA Generativa, estrutura os resultados em um modelo canônico, persiste os dados e compara duas apólices apresentando diferenças e evidências de origem.

A arquitetura foi desenhada como um fluxo multiagente. Cada componente possui uma responsabilidade explícita, reduzindo acoplamento e permitindo substituir OCR, modelo de linguagem, banco ou mecanismo de recuperação sem reescrever todo o sistema. O princípio adotado é: IA interpreta; código valida e compara determinísticamente sempre que possível.

- Leitura de PDF e imagens, com OCR gratuito como fallback.
- Uso de LLM local via Ollama, evitando dependência obrigatória de APIs pagas.
- Saída estruturada validada por JSON Schema/Pydantic.
- Rastreabilidade por página, evidência textual e score de confiança.
- Comparação A x B com classificação das diferenças.
- Interface Streamlit e geração de relatório comparativo em PDF.

2. Problema e objetivo

Apólices são documentos extensos, heterogêneos e frequentemente redigidos em linguagem jurídica. A comparação manual exige localizar coberturas, exclusões, franquias, limites de indenização, vigência e cláusulas especiais em diferentes pontos do documento. O objetivo do projeto é reduzir o tempo gasto nessa leitura, sem eliminar a necessidade de validação de especialistas.

2.1 Escopo do MVP

- Receber múltiplos arquivos PDF, PNG, JPG, JPEG, TIFF/TIF.
- Extrair texto nativo de PDF e aplicar OCR quando o texto for ausente ou insuficiente.
- Identificar dados gerais, coberturas, exclusões e cláusulas especiais.
- Persistir o resultado estruturado em SQLite.
- Comparar duas apólices e destacar equivalências, diferenças e itens exclusivos.
- Exibir evidências e páginas de origem.
- Gerar um PDF comparativo para demonstração.

3. Arquitetura da solução

PolicyCompare AI - Arquitetura do MVP

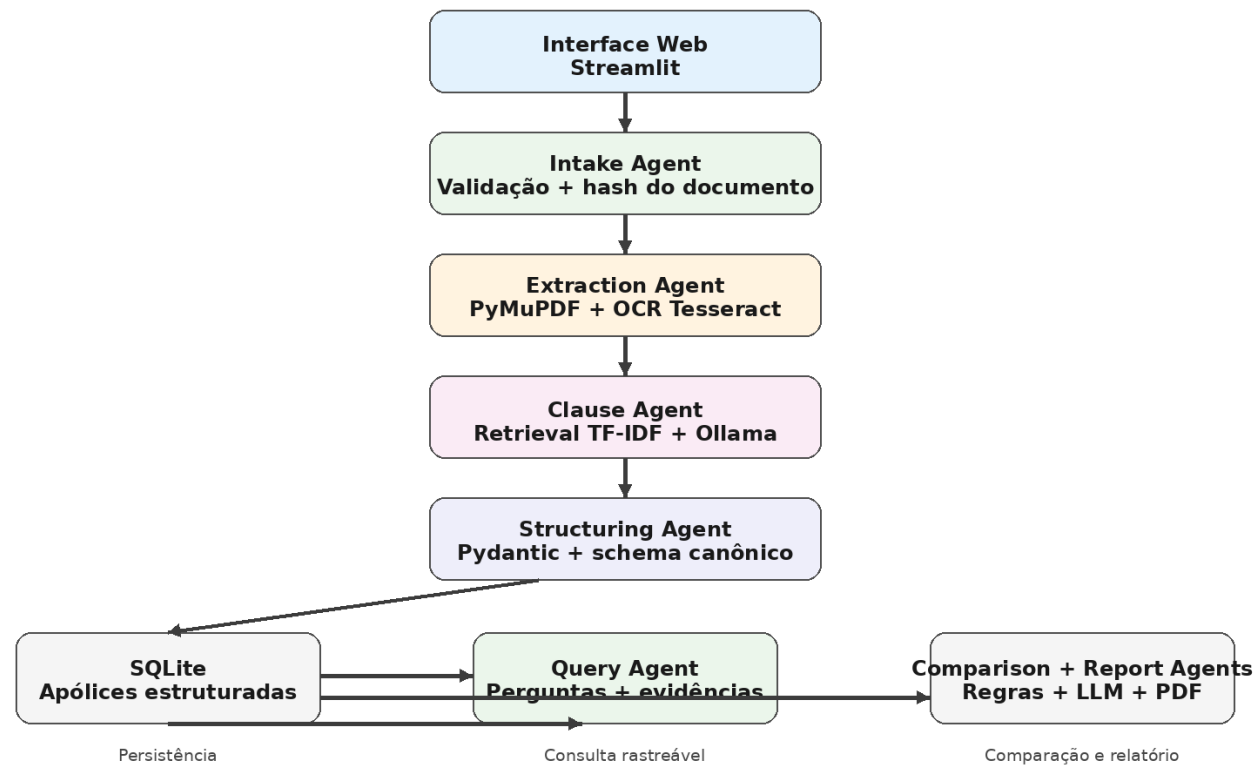


Figura 1 - Arquitetura lógica do MVP.

3.1 Separação de responsabilidades

Componente	Responsabilidade	Entrada	Saída
Intake Agent	Validação de formato, arquivo e identificação por hash.	PDF/Imagem	document_id
Extraction Agent	Extração de texto e OCR de páginas digitalizadas.	Documento	Texto por página
Chunking/Recovery	Divisão do texto e seleção de trechos relevantes.	Texto	Chunks ranqueados
Clause Agent	Interpretação semântica por LLM local.	Chunks	Dados de apólice
Structuring Agent	Validação e normalização do schema.	Dados do LLM	PolicyData
Repository	Persistência estruturada.	PolicyData	SQLite

Componente	Responsabilidade	Entrada	Saída
Query Agent	Perguntas sobre uma apólice estruturada com evidências.	Pergunta + PolicyData	Resposta + evidências
Comparison Agent	Comparação objetiva e semântica.	Apólice A e B	PolicyComparison
Report Agent	Relatório comparativo.	Comparação	PDF

4. Tecnologias utilizadas

Tecnologia	Uso	Justificativa
Python 3.11+	Back-end e agentes	Ecossistema maduro para IA, OCR, documentos e web.
Streamlit	Interface de demonstração	Permite entregar interface funcional com pouco código.
Ollama	Execução de LLM local	Privacidade, ausência de chave externa e troca simples de modelo.
Qwen local (configurável)	Extração semântica e resumo	Modelo local adequado ao protótipo; pode ser substituído por outro disponível no Ollama.
PyMuPDF	Leitura de PDF	Extração textual rápida e renderização de páginas.
Tesseract OCR	OCR gratuito	Fallback para imagens e PDFs digitalizados.
Pydantic	Schemas e validação	Contrato forte entre LLM e aplicação.
LangGraph	Orquestração	Workflow explícito com nós/agentes especializados.
SQLite	Persistência	Zero infraestrutura adicional para o MVP.
scikit-learn / TF-IDF	Recuperação local	Reduz o contexto enviado ao LLM sem exigir vector DB.
ReportLab	PDF comparativo	Geração programática do relatório final.

5. Descrição dos agentes

5.1 Intake Agent

Responsável pela porta de entrada do fluxo. Valida extensão, existência e tamanho do arquivo e produz identificador estável derivado de SHA-256.

- Evita processar formatos não suportados.
- Permite reconhecer o mesmo documento em execuções diferentes.

5.2 Extraction Agent

Tenta primeiro a extração textual nativa. Se uma página possuir pouco texto, renderiza a página e aplica OCR.

- PDF textual: PyMuPDF.
- Imagem/PDF digitalizado: Tesseract.
- Resultado preserva número da página e método de extração.

5.3 Clause Agent

Seleciona chunks potencialmente relevantes e solicita ao LLM local uma resposta estruturada.

- Não deve inventar campos ausentes.
- Cada cobertura/exclusão deve carregar evidência textual e página.
- A saída é validada contra schema Pydantic.

5.4 Structuring Agent

Converte a extração semântica para o modelo canônico PolicyData.

- Desacopla o schema persistido do formato bruto do LLM.
- Centraliza defaults, validações e tipos.

5.5 Query Agent

Responde perguntas em linguagem natural sobre os dados já extraídos de uma apólice.

- Usa exclusivamente o PolicyData como fonte.
- Retorna evidências existentes e sinaliza quando a resposta é inconclusiva.

5.6 Comparison Agent

Combina pareamento textual com regras determinísticas.

- Limite maior pode ser classificado como mais favorável.
- Franquia menor pode ser classificada como mais favorável.
- Itens presentes somente em uma apólice são destacados.
- Exclusões próximas são pareadas por similaridade.

5.7 Report Agent

Produz um documento comparativo contendo resumo, tabela de diferenças e aviso de validação humana.

- Entrega um artefato compartilhável.
- Evita que o resultado fique restrito à interface web.

6. Fluxo completo de processamento

1. Usuário envia uma ou mais apólices pela interface.
2. Intake Agent valida os arquivos e gera document_id.
3. Extraction Agent extrai texto por página; OCR é acionado quando necessário.
4. Chunking Service fragmenta o texto preservando página.
5. Retrieval Service usa TF-IDF para selecionar trechos relacionados a coberturas, exclusões, franquias, limites e dados gerais.
6. Clause Agent envia somente os trechos selecionados ao Ollama e solicita JSON estruturado.
7. Pydantic valida a resposta; o Structuring Agent cria o PolicyData.
8. PolicyRepository persiste o JSON no SQLite.
9. Usuário pode consultar uma apólice estruturada em linguagem natural por meio do Query Agent.
10. Usuário seleciona duas apólices na aba de comparação.
11. Comparison Agent pareia elementos e classifica diferenças.
12. O LLM produz um resumo executivo, com fallback determinístico se indisponível.
13. Report Agent gera o PDF comparativo e a interface apresenta evidências por página.

7. Modelo de dados e rastreabilidade

O schema canônico foi desenhado para não armazenar apenas a conclusão do modelo. Informações relevantes carregam Evidence com página, texto e confiança. Esse desenho permite auditoria, revisão humana e futuras métricas de qualidade.

Entidade	Campos principais
PolicyData	policy_id, document_name, insurer, policy_number, insured_name, product, vigência
Coverage	name, description, limit_value, currency, deductible_value, deductible_text, evidence
Exclusion	category, description, evidence
SpecialClause	category, title, description, evidence
Evidence	page, text, confidence
PolicyComparison	policy_a_id, policy_b_id, summary, items
ComparisonItem	dimension, item, A, B, status, rationale, evidence_a, evidence_b

8. Estratégia de comparação

A comparação não é delegada integralmente ao LLM. O MVP prioriza regras explícitas para reduzir variabilidade e facilitar testes.

Classificação	Significado
EQUIVALENTE	Condições equivalentes nos campos comparados.
MAIS_FAVORAVEL_A	Critério quantitativo favorece A, por exemplo

Classificação	Significado
	maior limite ou menor franquia.
MAIS_FAVORAVEL_B	Critério quantitativo favorece B.
SOMENTE_A	Item identificado somente na apólice A.
SOMENTE_B	Item identificado somente na apólice B.
DIFERENTE	Conteúdo presente em ambas, mas materialmente diferente.
INCONCLUSIVO	Dados insuficientes para uma conclusão segura.

9. Tratamento de erros, segurança e configuração

- Arquivo inválido ou vazio interrompe o fluxo antes do processamento de IA.
- Falhas de conexão ou timeout do Ollama são convertidas em erro controlado.
- A resposta estruturada do LLM é validada por Pydantic; JSON incompatível não é persistido silenciosamente.
- OCR é fallback e não é aplicado indiscriminadamente a todo PDF.
- Configurações ficam em .env; o arquivo real é ignorado pelo Git.
- O MVP não exige chaves de API externas e pode operar integralmente no computador do avaliador.
- Documentos reais podem conter dados sensíveis; para produção seriam necessários autenticação, controle de acesso, criptografia, retenção e trilha de auditoria.

10. Instalação e execução

- Instalar Python 3.11 ou 3.12.
- Criar ambiente virtual e executar `pip install -r requirements.txt`.
- Instalar Ollama e executar: `ollama pull qwen3:4b`.
- Instalar Tesseract OCR e o idioma português, garantindo `tesseract.exe` no PATH.
- Copiar `.env.example` para `.env`.
- Executar `python run.py` ou `streamlit run frontend/streamlit_app.py`.
- Abrir `http://localhost:8501`.

10.1 Roteiro de demonstração

- Criar os dois PDFs sintéticos pelo botão da barra lateral.
- Enviar os dois arquivos pela aba Processar.
- Acompanhar a estrutura JSON extraída.
- Abrir a aba Apólices e conferir coberturas/exclusões.
- Usar a aba Consultar para fazer uma pergunta e revisar evidências.
- Selecionar A e B na aba Comparar.
- Conferir classificação, evidências e páginas.
- Baixar o PDF comparativo.

11. Testes e validação

O pacote inclui testes automatizados para pontos determinísticos e foi submetido a smoke tests locais durante a geração deste entregável.

Teste	Resultado esperado
Intake ID	Mesmo arquivo produz o mesmo identificador.
Comparação de limite	Maior limite em A resulta em MAIS_FAVORAVEL_A.
PDF sintético	PyMuPDF extrai texto dos PDFs de demonstração.
Relatório comparativo	ReportLab gera PDF não vazio com a tabela de comparação.

Comando: `pytest -q`

12. Limitações conhecidas

- A ontologia é genérica; termos de diferentes ramos de seguro podem exigir especialização.
- Tabelas muito complexas, documentos com múltiplas colunas e baixa qualidade de digitalização podem reduzir a extração.
- TF-IDF não possui a mesma capacidade semântica de embeddings modernos.
- Modelos locais pequenos podem perder nuances jurídicas em cláusulas longas.
- A heurística de pareamento de coberturas usa similaridade textual e pode necessitar de taxonomia controlada.
- Não existem autenticação, multiusuário, RBAC, criptografia ou alta disponibilidade no MVP.
- O resultado deve ser validado por profissional qualificado e não constitui aconselhamento jurídico.

13. Evolução futura

- Substituir TF-IDF por embeddings multilíngues e Qdrant/pgvector.
- Adicionar taxonomia de coberturas e exclusões por ramo de seguro.
- Criar etapa human-in-the-loop para itens abaixo de um limiar de confiança.
- Adicionar OCR/layout especializado para tabelas, formulários e documentos escaneados.
- Executar extrações por página/seção em paralelo.
- Criar benchmark com apólices rotuladas e métricas precision, recall e F1 por campo.
- Versionar prompts, modelos e resultados para auditoria.
- Adicionar FastAPI, autenticação, trilha de auditoria e storage seguro.
- Adicionar exportação JSON/Excel e comparação de três ou mais apólices.
- Adicionar perguntas em linguagem natural com RAG e evidência clicável.

14. Estrutura do repositório

```
insurance-policy-ai/  
├─ app/  
│ └─ agents/  
│ └─ config/
```

```

| ├── models/
| ├── repositories/
| ├── services/
| └── workflows/
├── frontend/
├── tests/
├── data/
├── samples/
├── docs/
├── requirements.txt
├── docker-compose.yml
├── .env.example
└── README.md

```

15. Conclusão

O MVP atende aos requisitos centrais do desafio: recebe PDFs e imagens, extrai conteúdo, utiliza IA Generativa local, estrutura dados, persiste resultados, compara pelo menos duas apólices e apresenta diferenças em uma interface demonstrável. A arquitetura modular e multiagente deixa explícito onde cada responsabilidade reside e oferece um caminho direto para incorporar embeddings, OCR avançado, governança e integrações sem reconstruir a solução.

Apêndice A - Critérios de aceitação do MVP

Requisito	Atendimento
Leitura PDF/imagem	Sim - PyMuPDF + Tesseract
Extração automática	Sim - OCR + LLM
Dados estruturados	Sim - Pydantic/JSON
Comparação de 2 apólices	Sim
Diferenças ao usuário	Sim - tabela, status e evidências
IA Generativa	Sim - Ollama
Interface de demonstração	Sim - Streamlit
Agentes especializados	Sim - 7 agentes + workflow
Tratamento de erros	Sim - validações e exceções controladas
Credenciais ocultas	Sim - .env/.gitignore; por padrão sem API externa
Instruções de instalação	Sim - README